

Voice Stream

Стриминговый сервис распознавания голоса.

Смартфон захватывает микрофон через браузер и отправляет PCM-поток на Python-сервер, где faster-whisper транскрибирует речь в реальном времени.

Архитектура

Смартфон (браузер)

- └─ getUserMedia() → ScriptProcessor
 - └─ Int16 PCM chunks (16 kHz, моно)
 - └─ WebSocket → wss://192.168.x.x:8765
 - └─ Python asyncio server (fserver.py)
 - └─ faster-whisper (WhisperModel)
 - └─ JSON { type: "transcript", text: "..." }
 - └─ WebSocket → браузер (отображение)
 - └─ LLM API (OpenAI-совместимый)
 - └─ POST /api/chat
 - └─ WebSocket action "chat" → LLM → ответ
 - └─ Сохранение стенограмм
 - └─ transcripts/live_*.txt

Быстрый старт

1. Установка зависимостей

```
pip install -r requirements.txt
```

2. Запуск сервера

```
run_server.bat    # Windows  
# или  
python fserver.py
```

3. Узнать IP-адрес компьютера

```
# Windows
```

```
ipconfig
```

```
# macOS / Linux
```

```
ip addr # или ifconfig
```

4. Открыть веб-приложение на смартфоне

Сервер сам раздаёт HTML-интерфейс по HTTPS.

Откройте в браузере смартфона: `https://192.168.x.x:8765`

⚠️ Сервер автоматически генерирует самоподписанный SSL-сертификат.
Браузер покажет предупреждение — нажмите «Перейти на сайт» (или аналогично).

Возможности

Распознавание речи

- Запись микрофона через браузер, отправка PCM-потока по WebSocket
- Детекция тишины — распознавание запускается после паузы в речи
- Фильтрация галлюцинаций Whisper (блоклист фраз, контекстные эвристики)
- Автоматическое переподключение при разрыве соединения
- Возобновление записи после переподключения

Стенограммы

- Каждый распознанный сегмент сразу сохраняется в файл `transcripts/live_*.txt`
- Кнопка «Сохранить» — сохраняет текущую стенограмму в отдельный файл
- Кнопка «Очистить» — очищает текущую стенограмму
- Кнопка «Копировать» — копирует всю стенограмму в буфер обмена
- Просмотр, чтение и удаление сохранённых файлов через интерфейс

LLM чат (OpenAI-совместимый)

- Отправка сообщений через WebSocket в LLM (YandexGPT, OpenAI, Ollama и др.)
- Системный промпт (редактируемый, отправляется с каждым запросом)
- История сообщений сохраняется в течение сессии

- Кнопка «Очистить историю»
- **Управление системными промптами:** сохранение/редактирование/удаление промптов на сервере, загрузка промпта в чат одной кнопкой

Диалоговый режим (Dialog mode)

- Автоматическая отправка каждого распознанного сообщения в LLM
- Ответ LLM отображается в стенограмме как **«Агент»** и озвучивается через TTS
- Во время ожидания ответа или озвучивания новые фразы пропускаются
- Автоматически отключает режим «Ожидание» при включении

Кнопка «Вопрос» (Ask Question)

- Отправляет запрос **«Что дальше?»** в LLM
- Ответ отображается в стенограмме как **«Агент»** с пометкой источника (кнопка/автовопрос)
- Ответ автоматически озвучивается через TTS
- Таймаут ожидания ответа — 30 секунд

Режим «Ожидание» (Wait mode)

- Автоматический вопрос **«Что дальше?»** при паузе в речи пользователя
- Настраиваемый таймаут ожидания (1–30 секунд)
- Настраиваемый порог шума (10–500 RMS) — если уровень звука выше порога, таймер тишины сбрасывается
- Срабатывает один раз после каждого нового сообщения
- Недоступен при включённом диалоговом режиме

Режим «Наушники» (Headphones mode)

- При озвучивании TTS не приостанавливает распознавание речи
- Позволяет одновременно слушать ответ и продолжать говорить

Озвучивание текста (TTS)

Локальный синтез (браузерный)

- Встроенный браузерный синтезатор речи (SpeechSynthesis)
- Выбор голоса из доступных в системе (с приоритетом русских)
- Регулировка скорости речи

- Кнопки «Озвучить» / «Стоп»

Серверный синтез (Silerо)

- Синтез речи на Python-сервере через Silero TTS
- Выбор голоса: xenia (женский), aidar (мужской), baya (женский), kseniya (женский), eugeny (мужской)
- Воспроизведение PCM-аудио через WebSocket (chunk-by-chunk)
- Плавающая кнопка остановки TTS вне модального окна (если окно TTS закрыто, но озвучивание активно)

Тёмная тема

- Переключение между светлой и тёмной темой через кнопку в хедере

Роли в стенограмме

- «Гид» — распознанные сообщения пользователя
- «Агент» — ответы LLM с указанием источника: диалог, чат, автовопрос, кнопка «Вопрос»
- Все распознанные сообщения автоматически попадают в историю LLM чата

Настройка LLM

В файле `fserver.py` укажите параметры:

`LLM_API_URL = "https://api.openai.com/v1"`

`LLM_API_KEY = "your-api-key-here"`

`LLM_MODEL = "gpt-4o-mini"`

`# URL API (YandexGPT, OpenAI, Ollama...)`

`# API-ключ`

`# название модели`

Конфигурация **fserver.py**

Параметр	По умолчанию	Описание
MODEL_SIZE	medium	tiny / base / small / medium / large-v3
DEVICE	cuda	cpu / cuda
COMPUTE_TYPE	float16	int8 (CPU), float16 (GPU)
LANGUAGE	None	Язык распознавания (или None для авто)

Параметр	По умолчанию	Описание
SILENCE_THRESHOLD	2000	RMS для детекции тишины
SILENCE_CHUNKS	2	Чанков тишины до запуска транскрипции
PORT	8765	WebSocket + HTTP порт
TRANSCRIPTS_DIR	transcripts	Папка для сохранения стенограмм
LLM_API_URL	""	URL OpenAI-совместимого LLM API
LLM_API_KEY	""	Ключ доступа к LLM API
LLM_MODEL	""	Модель LLM

Модели Whisper (качество vs скорость на CPU)

Модель	Размер	Скорость (CPU)	Точность
tiny	39 MB	~1x реального	базовая
base	74 MB	~0.5x	хорошая
small	244 MB	~0.2x	отличная
medium	769 MB	медленно	высокая
large-v3	1550 MB	очень медленно	лучшая

На GPU с float16 — все модели работают быстрее реального времени.

Файлы

voice-stream/

- ├─ fserver.py ← Python WebSocket-сервер + faster-whisper + LLM API
- ├─ run_server.bat ← Скрипт запуска сервера (Windows)
- ├─ voice-recorder.html ← Веб-приложение для смартфона
- ├─ requirements.txt ← Python-зависимости
- ├─ README.md ← Эта документация
- ├─ cert.pem / key.pem ← Самоподписанный SSL-сертификат (генерируется автоматически)
- └─ transcripts/ ← Папка с сохранёнными стенограммами
 - └─ live_*.txt ← Файлы стенограмм